



BANGALORE INSTITUTE OF TECHNOLOGY

**DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
(IOT & CYBERSECURITY INCLUDING BLOCKCHAIN TECHNOLOGY)**



Project Presentation on

Adversarial Integration Tester

Automated Detection of Hidden Data Access in SaaS Integrations

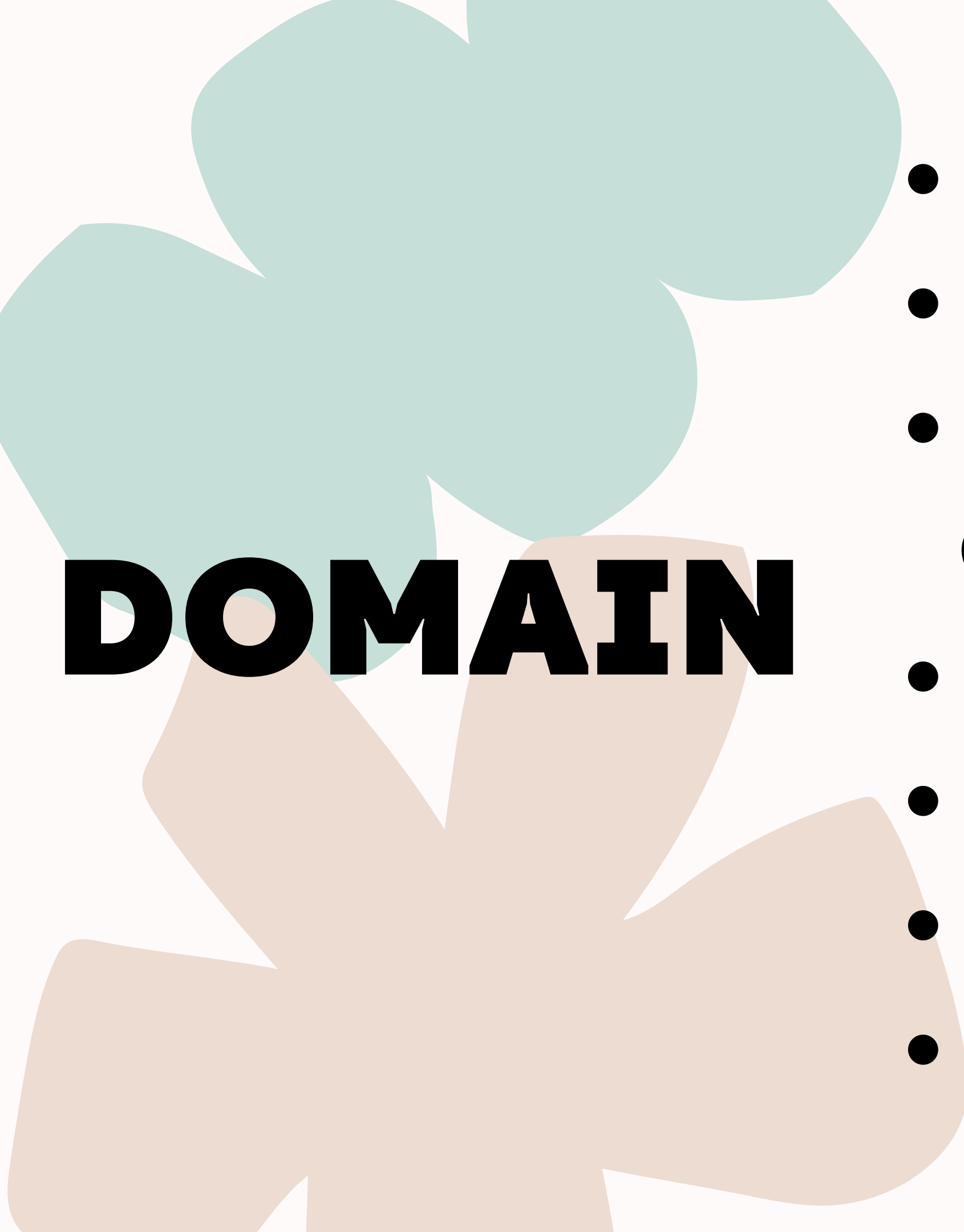
Major Project Phase - 1 Review - 1 (BIC685)

Under the Guidance of:

Apeksha Gowda
Assistant Professor
Dept. of CSE (ICB)
BIT, Bangalore - 560004

Presented By:

- **Adithya M N** (1BI23IC005)
- **Alaap V** (1BI23IC008)
- **Prajwal S** (1BI23IC037)
- **Nikhil Gowda T S** (1BI24IC402)



DOMAIN

- **Cloud Security**
- **Application Security**
- **Information Security**
- Governance**
- **Secure Software Engineering**
- **Data Security & Compliance**
- **OAuth & Identity Security**
- **DevSecOps**

PROBLEM STATEMENT

Modern enterprises rely on third-party SaaS integrations that use REST APIs and OAuth tokens to enable seamless data sharing and workflows. However, these integrations often have broad, static permissions, allowing access to more data than necessary. While organizations assume integrations only access permitted data, hidden API endpoints and over-permissioned tokens can expose sensitive information. The lack of automated mechanisms to verify actual runtime behavior makes security audits largely manual and incomplete, increasing the risk of unnoticed data access.

OBJECTIVES

To design and develop an automated adversarial testing framework that detects hidden or unauthorized data access in third-party SaaS integrations.

Differential API Fuzzing

- Automated coverage-guided request mutation testing
- State-aware sequential REST endpoint validation
- Cross-version behavioral response comparison analysis

Taint Analysis / Data-Flow Tracking

- Synthetic markers to trace data flow paths
- Continuous tracking of sensitive data propagation
- Detection of unauthorized or unintended data access

Behavioral Divergence Analysis

- Comparison of expected and observed API behavior
- Identification of hidden or unintended access paths

EXPECTED DELIVERABLES

Functional Prototype System

- Automated adversarial testing framework
- REST API fuzzing engine
- Taint-seeded data injection module
- Behavioral divergence detection system

Minimal-Access Verification Report

- APIs accessed by integration
- Data fields retrieved
- Detection of hidden/synthetic field access
- Risk classification score

Test Case Evaluation Results

- Testing on sample SaaS integration scenarios
- Comparison of declared permissions versus observed behavior
- Detection accuracy analysis

LITERATURE REVIEW

SI No	Author(s)	Title (Year)	Methodology	Metrics	Results	Limitations	Problem Identified	Conclusion
1	Zhang et al.	MINER (2023)	Hybrid fuzzing, static + runtime, ML parameter prediction	Pass rate, error count, coverage	+23% pass rate, +97% errors	No cross-version analysis, no data-leak detection	Poor deep-state exploration, parameter misuse	Better sequence generation, limited security analysis
2	Wang et al.	foREST (2022)	Tree-based dependency modeling, stateful fuzzing	Code coverage, bug count	11.5–82.5% coverage, 11 bugs	No privacy/taint tracking	Dependency explosion in graphs	Efficient dependency handling, limited security scope
3	Zhou et al.	Pythia (2020)	Grammar-based fuzzing, coverage-guided, learning	Coverage, bug discovery	29 bugs, improved coverage	No cross-environment analysis	Lack of semantic mutation	Better semantic fuzzing, single-system focus
4	WuppieFuzz Team	WuppieFuzz (2025)	Stateful fuzzing, sequence mutation, OpenAPI-based	Coverage, endpoint exploration	Improved state exploration	No data-access / abuse detection	Automation challenges in API fuzzing	Good automation, weak security validation
5	Various	Differential API Fuzzing (2025)	Differential testing, response comparison	Behavioral divergence	Detects inconsistencies	No SaaS/privacy focus	Hidden API behavior differences	Useful for inconsistency, lacks data-flow validation

LITERATURE REVIEW

SI No	Author(s)	Title (Year)	Methodology	Metrics	Results	Limitations	Problem Identified	Conclusion
1	PoPETs	Privacy Leakage Detection (2025)	Data-flow tracking, cross-app analysis, rule extraction	Leakage rate, inference probability	Hidden leakage detection, cross-app chains (~7.67%)	App-level focus, no API integration	Cross-app leakage, hidden flows	Effective detection, limited ecosystem scope
2	ArXiv (2502.21026)	Advanced Taint Tracking (2025)	Taint tracking, distributed systems, flow tracing	Leakage detection accuracy, flow coverage	Hidden data flow identification	No fuzzing integration, no automation	Undetected hidden flows	Strong tracking, lacks testing automation
3	ArXiv (2409.10411)	Data-flow Analysis Framework (2024)	Static/dynamic analysis, cloud services, data	Sensitivity detection, misuse rate	Sensitive data misuse identification	No adversarial/fuzz testing	Cloud data misuse	Good analysis, lacks attack simulation
4	API Financial Data Privacy	API Privacy Study (2024)	Survey, vulnerability analysis, SaaS APIs	Risk categorization, vulnerability types	Identified integration risks	No implementation, no automation	SaaS API exposure risks	Awareness-focused, no validation
5	Cloud API Security	API Security Taxonomy (2024–2025)	Taxonomy design, mitigation strategies, classification	Threat categories, mitigation mapping	Structured API security landscape	Defensive-only, no fuzzing/taint validation	Lack of proactive testing	Strong taxonomy, limited practical validation

LITERATURE REVIEW

SI No	Author(s)	Title (Year)	Methodology	Metrics	Results	Limitations	Problem Identified	Conclusion
1	Foley & Maffeis	APIRL (2024)	Reinforcement learning, request mutation, transformer	Bug count, coverage, request efficiency	Higher bug detection, fewer requests	No taint tracking, no behavior analysis	Inefficient random fuzzing	Intelligent fuzzing, limited security depth
2	Arcuri et al.	White-box REST Fuzzing (2023)	White-box heuristics, search-based fuzzing,	Code coverage, fault detection	Improved coverage, better schema handling	No privacy analysis, no integration behavior	Under-specified schemas, low coverage	Better coverage, lacks security focus
3	Alshammari et al.	API Threat Detection (2024)	Behavioral monitoring, anomaly detections real-	Detection accuracy, anomaly rate	Real-time threat detection	No fuzzing, no adversarial testing	Hidden API threats undetected	Good monitoring, not proactive
4	Seran et al.	Dynamic Taint Analysis (2024)	Dynamic taint tracking, API testing, data-flow tracing	Data-flow accuracy, fault detection	Sensitive data tracking improved	No differential fuzzing, no SaaS focus	Hidden data flow misuse	Strong tracking, limited scope
5	OpenText Security Team	API Security Trends (2024)	Industry analysis, trend study, risk assessment	Risk categories, trend metrics	Identified rising API risks	No implementation, no validation framework	Lack of automated testing	Awareness-focused, no practical solution

SOCIAL IMPACT

In today's interconnected digital environment, users and organizations rely heavily on multiple applications that share data through APIs, often without full visibility into how that data is used. By automatically detecting unauthorized or excessive data access, our system helps protect user privacy, prevent sensitive data leakage, and ensure that digital services operate within their intended boundaries.

From an organizational and societal perspective, the project promotes trust, transparency, and accountability in software systems. It enables companies to verify whether integrations behave as expected, reducing risks of data breaches and ensuring compliance with data protection regulations. By moving from manual audits to automated, continuous testing, your approach strengthens overall cybersecurity posture and supports safer adoption of cloud-based services.

THANK

YOU